Documento Técnico de Arquitectura

Microservicio Transversal de IA — Gestión de Solicitudes BPO

Autor: Manuel Sebastián Torres Hernández **Fecha:** 23 de marzo de 2026

1. Resumen Ejecutivo

El microservicio BPO-AI automatiza los pasos 1 al 6 del proceso de gestión de solicitudes: validación semántica, clasificación, asignación de prioridad, generación de justificación, decisión del siguiente paso y creación del caso en plataformas externas. Está diseñado para operar de forma transversal sobre múltiples empresas cliente, cada una con sus propias categorías, reglas de prioridad e integraciones.

Decisión estratégica: Se adoptó un enfoque híbrido LLM + reglas de negocio configurables. El LLM gestiona la comprensión del lenguaje natural (validación, clasificación, justificación); las reglas deterministas del YAML controlan prioridad y enrutamiento. Esto garantiza interpretabilidad, auditabilidad y adaptabilidad por cliente sin depender del modelo para decisiones de negocio críticas.

2. Arquitectura de la Solución

2.1 Justificación del enfoque híbrido

Un modelo puramente basado en reglas no puede manejar texto libre con variabilidad lingüística. Un LLM puro para prioridad y enrutamiento sería opaco, no auditable y susceptible a alucinaciones en decisiones de negocio. El enfoque híbrido asigna cada responsabilidad al mecanismo más adecuado:

Paso	Mecanismo	Razón
Validación semántica	LLM	Comprende texto libre, extrae entidades
Clasificación	LLM + validación post-modelo	Entiende intención, pero se restringe a categorías del YAML
Prioridad	Reglas YAML / servicio externo	Auditable, determinístico, cambiable por cliente
Justificación	LLM	Redacción en lenguaje natural
Siguiente paso	Reglas YAML	Determinístico, contrato contractual con el cliente
Caso externo	Adaptador por empresa	Desacoplado de la lógica de negocio

2.2 Estructura de componentes

El sistema se organiza en cuatro capas desacopladas:

Capa API (routes.py): Punto de entrada único vía REST. Recibe el JSON, valida el esquema con Pydantic y delega al pipeline. Maneja errores HTTP de forma centralizada (empresa no registrada → 404, error interno → 500).

Capa Pipeline (pipeline.py): Orquesta los 6 pasos de forma secuencial usando el patrón *Context Object*: un objeto `ContextoPipeline` viaja por cada paso acumulando resultados. Implementa cortocircuito en el paso 1 — si la solicitud es inválida, retorna inmediatamente sin consumir llamadas adicionales al LLM.

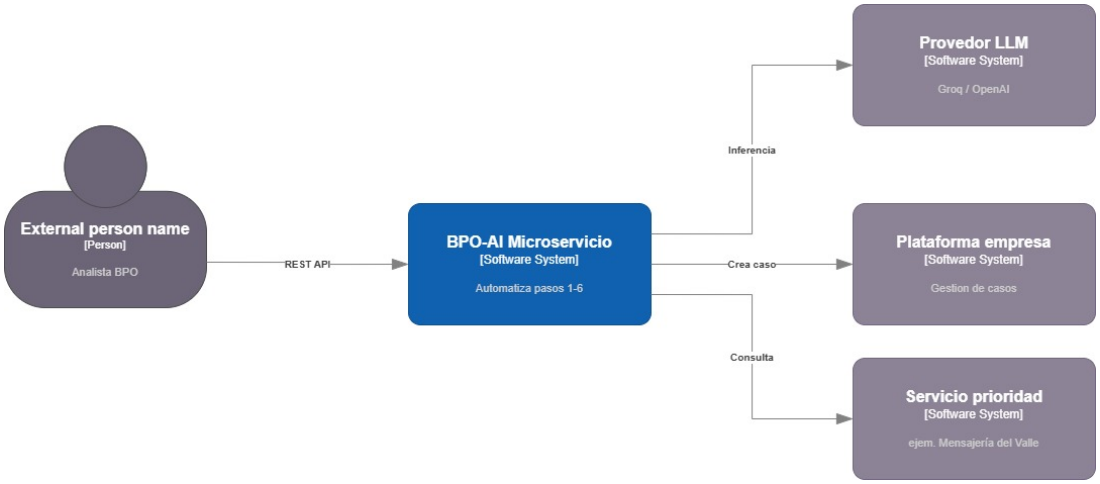
Capa de Configuración (config/loader.py + YAMLs): Cada empresa tiene un archivo YAML independiente con sus categorías, reglas de prioridad, delegaciones y plataforma externa. El loader usa `lru_cache` para evitar lecturas repetidas a disco. **Agregar una nueva empresa no requiere modificar ningún archivo Python**, solo crear un YAML nuevo.

Capa de Integración (integrations/): Implementa el patrón adaptador sobre una interfaz abstracta `PlataformaExternaBase`. El `platform_registry` mapea el tipo de plataforma configurado en el YAML a su clase concreta. Agregar soporte para una nueva plataforma (Salesforce, ServiceNow) implica únicamente crear una clase que implemente la interfaz y registrarla.

3. Arquitectura del Sistema (Modelo C4)

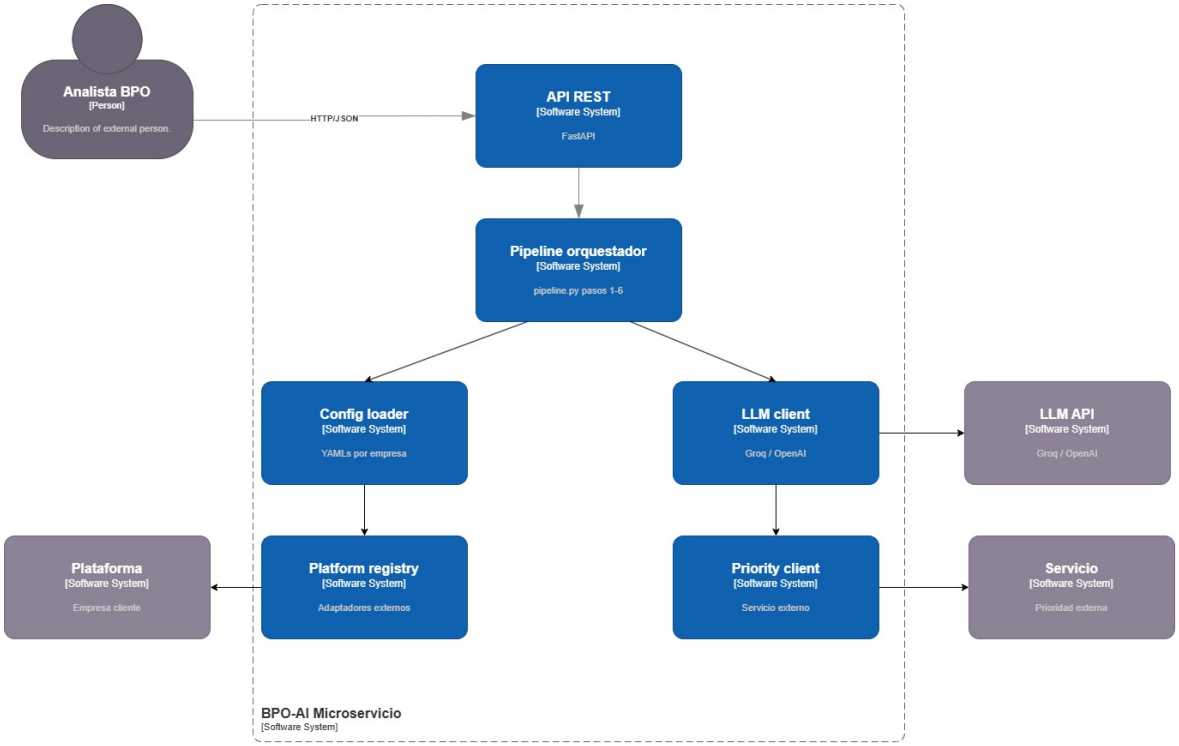
La arquitectura fue documentada siguiendo el estándar C4, que permite describir el sistema en niveles progresivos de detalle.

3.1 Nivel 1 — Contexto del Sistema



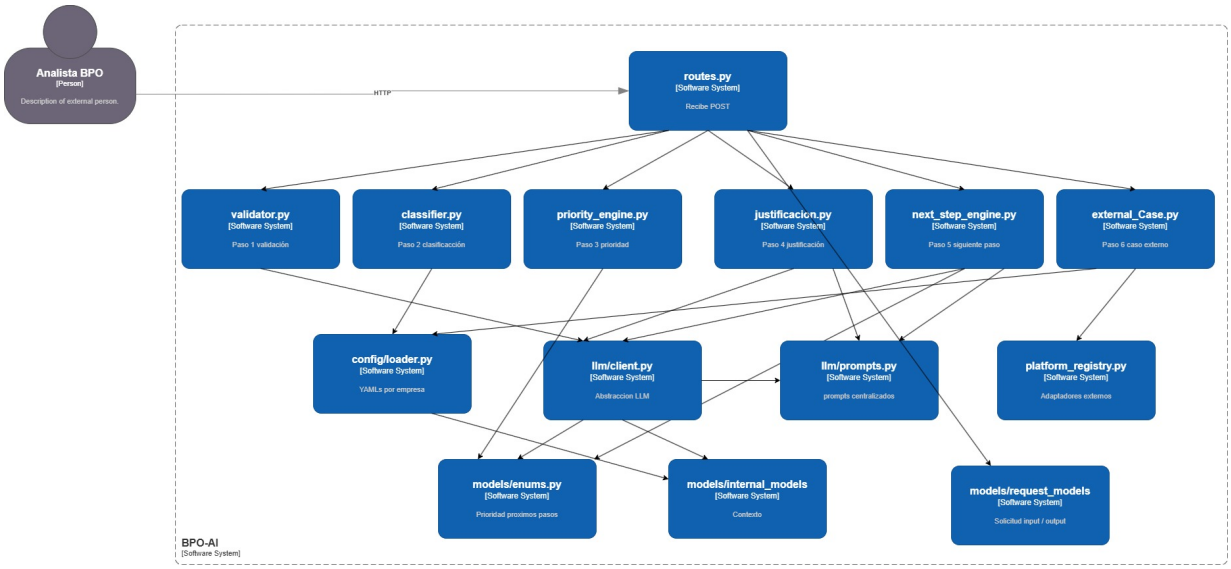
El microservicio actúa como punto de entrada único para los analistas BPO. Se integra con tres sistemas externos: el proveedor LLM para inferencia semántica, las plataformas de atención de cada empresa cliente para la creación de casos, y los servicios externos de prioridad que algunos clientes proveen directamente.

3.2 Nivel 2 — Contenedores



Los contenedores internos están completamente desacoplados entre sí: el pipeline no conoce los detalles del proveedor LLM ni de la plataforma externa. Toda comunicación ocurre a través de interfaces abstractas, lo que permite reemplazar cualquier componente sin afectar al resto.

3.3 Nivel 3 — Componentes



Cada paso del pipeline es un módulo Python independiente con una única responsabilidad. Los módulos de infraestructura compartida (`llm/client.py`, `config/loader.py`, `platform_registry`) son consumidos por múltiples pasos pero no pertenecen a ninguno, evitando acoplamiento cruzado.

4. Escalabilidad y Adaptabilidad

¿Por qué esta arquitectura soporta el crecimiento? La variabilidad entre empresas está externalizada en archivos de configuración, no en el código. El código central es agnóstico a los detalles de cada cliente.

- **Nueva empresa:** Crear un archivo `app/config/companies/<nombre>.yaml`. Sin cambios en Python.
- **Nueva categoría:** Agregar una línea al YAML de la empresa correspondiente.
- **Cambio de reglas de prioridad:** Modificar las reglas en el YAML. El motor las lee en cada request (con cache).
- **Nueva plataforma externa:** Implementar `PlataformaExternaBase` y registrar en `platform_registry.py`.
- **Nuevo proveedor LLM:** Implementar `LLMClient` y agregar al factory. Configurable por variable de entorno.
- **Servicio externo de prioridad:** Activar `prioridad_externa.activa: true` en el YAML del cliente y configurar la URL como variable de entorno.
- **Mayor volumen:** El diseño stateless de la API permite escalar horizontalmente añadiendo réplicas sin coordinación entre instancias.

5. Manejo de Errores

- **Plataforma externa falla:** El adaptador captura la excepción, registra el error en logs estructurados y retorna el caso con estado `PENDIENTE_REINTENTO` y flag `plataforma_error: true`. El endpoint `/solicitudes/health` permite monitorear el estado del servicio. En producción, este flag alimentaría un sistema de alertas (CloudWatch, Datadog).
- **Casos duplicados:** El campo `solicitud_id` actúa como idempotency key. En producción se implementaría verificación contra Redis o la base de datos antes de procesar, retornando el resultado cacheado sin reprocesar.
- **Empresa no parametrizada:** El `config/loader.py` detecta la ausencia del YAML y lanza `CompanyNotFoundError`, que la capa API convierte en un HTTP 404 con la lista de empresas disponibles, sin que el error llegue al pipeline.

- **LLM no disponible:** Cada paso que usa LLM implementa un fallback determinístico: el validador acepta por longitud mínima, el clasificador asigna “Otro”, y el justificador genera un texto genérico con los datos disponibles.

6. Personalización por Empresa

El caso de **Mensajería del Valle** está implementado. En su YAML se configura `prioridad_externa.activa: true` y `endpoint_env_var: MENSAJERIA_VALLE_PRIORITY_URL`. El `priority_engine.py` detecta esta configuración y llama al microservicio del cliente enviando `tipo_documento`, `numero_documento` y `tipo_solicitud`. Si el servicio externo falla por cualquier razón (timeout, error HTTP, respuesta inválida), el motor hace fallback automático a las reglas locales del YAML, garantizando que el pipeline siempre produce un resultado.

7. Propuesta de Despliegue en la Nube (AWS)

Componente	Servicio AWS	Justificación
API REST	API Gateway + ECS Fargate	Escalado automático, sin gestión de servidores
Configuración empresas	S3 + Parameter Store	YAMLs versionados, cambios sin redeploy
Secretos (API keys)	Secrets Manager	Rotación automática, sin variables en código
Cola de solicitudes	SQS	Desacopla ingesta de procesamiento en picos de volumen
Logs y métricas	CloudWatch	Alertas sobre `plataforma_error`, latencia LLM
Cache duplicados	ElastiCache (Redis)	Verificación de `solicitud_id` en $<$1ms
CI/CD	CodePipeline + ECR	Build, test y deploy automatizados por rama

8. Conclusión

La solución entregada automatiza los 6 pasos del proceso BPO con una arquitectura que separa claramente la lógica de negocio (en YAMLs por empresa), la inteligencia artificial (en el cliente LLM), y las integraciones externas (en adaptadores intercambiables). Esta separación garantiza que el crecimiento en número de empresas, categorías o plataformas se gestione mediante configuración, no mediante cambios de código, cumpliendo directamente con los criterios de escalabilidad y adaptabilidad evaluados.